Propiedades

Container:	Modules	DateCreated:	3/2/2015 21:03:52
LastUpdated:	3/2/2015 21:03:52	Owner:	admin
UserName:	admin		

Código

```
Attribute VB_Name = "bas_Roles"
Option Compare Database

Public Function Get_Rol(intRol As Integer) As String

Dim dbs As DAO.Database
Dim rst As DAO.Recordset

Dim strSQL As String

   strSQL = "SELECT dbo_Roles.ID_Rol, dbo_Roles.Rol " & _
            "FROM dbo_Roles " & _
            "WHERE (((dbo_Roles.ID_Rol)=" & intRol & "));"

   Set dbs = CurrentDb()
   Set rst = dbs.OpenRecordset(strSQL, dbOpenDynaset)

   ' Si no hay registros...
   If rst.RecordCount = 0 Then Exit Function

   ' Devolvemos el rol del usuario
   Get_Rol = rst!Rol

   ' Cerramos el Recordset
   rst.Close

   ' Borramos los objetos
   Set rst = Nothing
   Set dbs = Nothing

End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	3/2/2015 21:03:52
LastUpdated:	3/2/2015 21:03:52	Owner:	admin
UserName:	admin		

Código

```
Attribute VB_Name = "bas_Seguridad_Sesion"
Option Compare Database
Public Sub Block_Usuario(intUsuario As Integer)

Dim dbs As DAO.Database
Dim rst As DAO.Recordset

Dim strSQL As String

  strSQL = "SELECT dbo_Usuarios.Activo " & _
           "FROM dbo_Usuarios " & _
           "WHERE ID_Usuario = " & intUsuario & ";"

  Set dbs = CurrentDb()
  Set rst = dbs.OpenRecordset(strSQL, dbOpenDynaset)

  With rst

    ' Modificamos el registro
    .Edit

    .Fields("Activo").Value = False

    ' Actualizamos el registro
    .Update

  End With

  ' Cerramos el recordset
  rst.Close
  ' Borramos los objetos
  Set rst = Nothing
  Set dbs = Nothing

End Sub

```

```
Public Function Get_ErrMaxLogin() As Byte

Dim dbs As DAO.Database
Dim rst As DAO.Recordset

Dim strSQL As String

  strSQL = "SELECT dbo_Opciones.ID_Opcion, " & _
               "dbo_Opciones.ErrMaxLogin " & _
           "FROM dbo_Opciones " & _
           "WHERE (((dbo_Opciones.ID_Opcion) = 1));"

  Set dbs = CurrentDb()
  Set rst = dbs.OpenRecordset(strSQL, dbOpenDynaset)

  With rst

    ' Devolvemos 0 por que no hay registros
    If .RecordCount = 0 Then Get_ErrMaxLogin = 0

    ' Devolvemos el número máximo de intentos
    Get_ErrMaxLogin = !ErrMaxLogin

  End With

  ' Cerramos el recordset
  rst.Close
  ' Borramos los objetos
  Set rst = Nothing
  Set dbs = Nothing

End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	3/2/2015 21:03:52
LastUpdated:	3/2/2015 21:03:52	Owner:	admin
UserName:	admin		

Código

```
Attribute VB_Name = "bas_Sesion"
Option Compare Database

Public Function Log_Sesion(intUsuario As Integer, _
                    strResultado As String, _
                    Optional strContraseña As String = "")

Dim dbs As DAO.Database
Dim rst As DAO.Recordset

  Set dbs = CurrentDb()
  Set rst = dbs.OpenRecordset("dbo_Log_Sesion")

  With rst

    ' Agregamos un registro
    .AddNew

    ' Especificamos los campos en los que vamos a insertar
    ' los datos relativos a la sesión de usuario.
    .Fields("Fecha").Value = Now()
    '.Fields("Terminal").Value = Environ("Computername")
    .Fields("Terminal").Value = CreateObject("WScript.Network").ComputerName

    .Fields("ID_Usuario").Value = intUsuario
    .Fields("Resultado").Value = strResultado
    .Fields("ContraseñaErronea").Value = strContraseña

    ' Actualizamos el registro
    .Update

  End With

  ' Cerramos el recordset
  rst.Close

```

```
    ' Borramos los objetos
    Set rst = Nothing
    Set dbs = Nothing

End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	3/2/2015 21:03:52
LastUpdated:	3/2/2015 21:03:52	Owner:	admin
UserName:	admin		

Código

```
1   Attribute VB_Name = "bas_Usuarios"
2   Option Compare Database
3    '-----------------------------------------------------------------
4   ' Procedure : Get_Contraseña
5   ' Author    : Omar Dominguez
6   ' Date      : 22/09/2014
7   ' Purpose   : Obtiene la contraseña del usuario pasado como argumento.
8   '-----------------------------------------------------------------
9   '
10  Public Function Get_Contraseña(intUsuario As Integer) As String
11
12  Dim dbs As DAO.Database
13  Dim rst As DAO.Recordset
14
15  Dim strSQL As String
16
17     strSQL = "SELECT dbo_Usuarios.Contraseña " & _
18              "FROM dbo_Usuarios " & _
19              "WHERE ID_Usuario = " & intUsuario & ";"
20
21     Set dbs = CurrentDb()
22     Set rst = dbs.OpenRecordset(strSQL, dbOpenDynaset)
23
24  Dim strContraseña As String
25
26     If rst.RecordCount <> 0 Then
27
28        strContraseña = rst!Contraseña
29
30     End If
31
32     ' Devolvemos la contraseña
33     Get_Contraseña = strContraseña
34
35     ' Cerramos el recordset
36     rst.Close
```

```
37
38      ' Borramos los objetos
39      Set rst = Nothing
40      Set dbs = Nothing
41
42  End Function
43
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	8/12/2015 14:58:19
LastUpdated:	8/12/2015 14:58:19	Owner:	admin
UserName:	admin		

Código

```
VERSION 1.0 CLASS
BEGIN
  MultiUse = -1  'True
END
Attribute VB_Name = "clFormWindow"
Attribute VB_GlobalNameSpace = False
Attribute VB_Creatable = False
Attribute VB_PredeclaredId = False
Attribute VB_Exposed = False
Option Compare Database
Option Explicit


'*************************************************************
' Class module: clFormWindow                                 *
'*************************************************************
' Moves and resizes a window in the coordinate system       *
' of its parent window.                                      *
' N.B.: This class was developed for use on Access forms    *
'      and has not been tested for use with other window    *
'      types.                                                *
'*************************************************************



'*************************************************************
' Type declarations
'*************************************************************

Private Type RECT      'RECT structure used for API calls.
    Left As Long
    Top As Long
    Right As Long
    Bottom As Long
End Type

```

```
37
38  Private Type POINTAPI   'POINTAPI structure used for API calls.
39      X As Long
40      Y As Long
41  End Type
42
43
44
45  '*************************************************************
46  ' Member variables
47  '*************************************************************
48
49  Private m_hWnd As Long          'Handle of the window.
50  Private m_rctWindow As RECT     'Rectangle describing the sides of the last polled location of the v
51
52
53
54  '*************************************************************
55  ' Private error constants for use with RaiseError procedure
56  '*************************************************************
57
58  Private Const m_ERR_INVALIDHWND = 1
59  Private Const m_ERR_NOPARENTWINDOW = 2
60
61
62
63  '*************************************************************
64  ' API function declarations
65  '*************************************************************
66
67
68
69
70  '*************************************************************
71  ' Private procedures
72  '*************************************************************
73
74  Private Sub RaiseError(ByVal lngErrNumber As Long, ByVal strErrDesc As String)
75  'Raises a user-defined error to the calling procedure.
76
77      Err.Raise vbObjectError + lngErrNumber, "clFormWindow", strErrDesc
78
79  End Sub
80
```

```
Private Sub UpdateWindowRect()
'Places the current window rectangle position (in pixels, in coordinate system of parent window) i
m_rctWindow.

  Dim ptCorner As POINTAPI

  If m_hWnd = 0 Or apiIsWindow(m_hWnd) Then
    apiGetWindowRect m_hWnd, m_rctWindow   'm_rctWindow now holds window coordinates i
coordinates.

    If Not Me.Parent Is Nothing Then
      'If there is a parent window, convert top, left of window from screen coordinates to parent
coordinates.
      With ptCorner
        .X = m_rctWindow.Left
        .Y = m_rctWindow.Top
      End With

      apiScreenToClient Me.Parent.hWnd, ptCorner

      With m_rctWindow
        .Left = ptCorner.X
        .Top = ptCorner.Y
      End With

      'If there is a parent window, convert bottom, right of window from screen coordinates to p
window coordinates.
      With ptCorner
        .X = m_rctWindow.Right
        .Y = m_rctWindow.Bottom
      End With

      apiScreenToClient Me.Parent.hWnd, ptCorner

      With m_rctWindow
        .Right = ptCorner.X
        .Bottom = ptCorner.Y
      End With
    End If
  Else
    RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
  End If

End Sub
```

```
123
124
125
126 '*************************************************************
127 ' Public read-write properties
128 '*************************************************************
129
130 Public Property Get hWnd() As Long
131 'Returns the value the user has specified for the window's handle.
132
133    If m_hWnd = 0 Or apiIsWindow(m_hWnd) Then
134       hWnd = m_hWnd
135    Else
136       RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
137    End If
138
139 End Property
140
141
142 Public Property Let hWnd(ByVal lngNewValue As Long)
143 'Sets the window to use by specifying its handle.
144 'Only accepts valid window handles.
145
146    If lngNewValue = 0 Or apiIsWindow(lngNewValue) Then
147       m_hWnd = lngNewValue
148    Else
149       RaiseError m_ERR_INVALIDHWND, "The value passed to the hWnd property is not a valid win
    handle."
150    End If
151
152 End Property
153
154 '-----------------------------------------------------
155
156 Public Property Get Left() As Long
157 'Returns the current position (in pixels) of the left edge of the window in the coordinate system of
    parent window.
158
159    If m_hWnd = 0 Or apiIsWindow(m_hWnd) Then
160       UpdateWindowRect
161       Left = m_rctWindow.Left
162    Else
163       RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
164    End If
165
```

```
End Property


Public Property Let Left(ByVal lngNewValue As Long)
'Moves the window such that its left edge falls at the position indicated
'(measured in pixels, in the coordinate system of its parent window).

  If m_hWnd = 0 Or apiIsWindow(m_hWnd) Then
    UpdateWindowRect
    With m_rctWindow
      apiMoveWindow m_hWnd, lngNewValue, .Top, .Right - .Left, .Bottom - .Top, True
    End With
  Else
    RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
  End If

End Property

'-----------------------------------------------------

Public Property Get Top() As Long
'Returns the current position (in pixels) of the top edge of the window in the coordinate system of
parent window.

  If m_hWnd = 0 Or apiIsWindow(m_hWnd) Then
    UpdateWindowRect
    Top = m_rctWindow.Top
  Else
    RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
  End If

End Property


Public Property Let Top(ByVal lngNewValue As Long)
'Moves the window such that its top edge falls at the position indicated
'(measured in pixels, in the coordinate system of its parent window).

  If m_hWnd = 0 Or apiIsWindow(m_hWnd) Then
    UpdateWindowRect
    With m_rctWindow
      apiMoveWindow m_hWnd, .Left, lngNewValue, .Right - .Left, .Bottom - .Top, True
    End With
  Else
    RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
```

```
210     End If
211
212   End Property
213
214   '-----------------------------------------------------
215
216   Public Property Get Width() As Long
217   'Returns the current width (in pixels) of the window.
218
219     If m_hWnd = 0 Or apiIsWindow(m_hWnd) Then
220       UpdateWindowRect
221       With m_rctWindow
222         Width = .Right - .Left
223       End With
224     Else
225       RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
226     End If
227
228   End Property
229
230
231   Public Property Let Width(ByVal lngNewValue As Long)
232   'Changes the width of the window to the value provided (in pixels).
233
234     If m_hWnd = 0 Or apiIsWindow(m_hWnd) Then
235       UpdateWindowRect
236       With m_rctWindow
237         apiMoveWindow m_hWnd, .Left, .Top, lngNewValue, .Bottom - .Top, True
238       End With
239     Else
240       RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
241     End If
242
243   End Property
244
245   '-----------------------------------------------------
246
247   Public Property Get Height() As Long
248   'Returns the current height (in pixels) of the window.
249
250     If m_hWnd = 0 Or apiIsWindow(m_hWnd) Then
251       UpdateWindowRect
252       With m_rctWindow
253         Height = .Bottom - .Top
```

```
254         End With
255     Else
256         RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
257     End If
258
259 End Property
260
261
262 Public Property Let Height(ByVal lngNewValue As Long)
263 'Changes the height of the window to the value provided (in pixels).
264
265     If m_hWnd = 0 Or apiIsWindow(m_hWnd) Then
266         UpdateWindowRect
267         With m_rctWindow
268             apiMoveWindow m_hWnd, .Left, .Top, .Right - .Left, lngNewValue, True
269         End With
270     Else
271         RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
272     End If
273
274 End Property
275
276
277
278 '*************************************************************
279 ' Public read-only properties
280 '*************************************************************
281
282 Public Property Get Parent() As clFormWindow
283 'Returns the parent window as a clFormWindow object.
284 'For forms, this should be the Access MDI window.
285
286     Dim fwParent As New clFormWindow
287     Dim lngHWnd As Long
288
289     If m_hWnd = 0 Then
290         Set Parent = Nothing
291     ElseIf apiIsWindow(m_hWnd) Then
292         lngHWnd = apiGetParent(m_hWnd)
293         fwParent.hWnd = lngHWnd
294         Set Parent = fwParent
295     Else
296         RaiseError m_ERR_INVALIDHWND, "The window handle " & m_hWnd & " is no longer valid."
297     End If
```

```
298
299     Set fwParent = Nothing
300
301 End Property
302
303
304
305
306
307
308
309
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	3/2/2015 21:03:52
LastUpdated:	3/2/2015 21:03:52	Owner:	admin
UserName:	admin		

Código

```
1 Attribute VB_Name = "LimitarTamañoCampo"
2 Option Compare Database
3 Option Explicit
4 Public Const SW_SHOW = 1
5
6 Function LimitaCaracteres(ControlTexto As TextBox, ByVal NumeroMax As Long) As Boolean
7    LimitaCaracteres = Len(Nz(ControlTexto.Text, "")) < NumeroMax
8 End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	3/2/2015 21:03:52
LastUpdated:	3/2/2015 21:03:52	Owner:	admin
UserName:	admin		

Código

```
Attribute VB_Name = "mdlAvisos"
Option Compare Database
Option Explicit

'*******************************************************************************
'* ActualizaAvisos
'* Realiza la Actualización de las Fechas de Avisos, del estado de NumeroAviso
'* correspondiente a cada aviso y del campo boleano Avisar que es el que indica
'* si toca o no emitir un aviso.
'* Argumentos:
'* uso:  Call ActualizaAvisos

'*******************************************************************************

Function ActualizaAvisos()
Dim dbs As DAO.Database, rst As DAO.Recordset, rst1 As DAO.Recordset
Dim i As Integer

Set dbs = CurrentDb

'Abrimos un Recordset de la tabla que contiene la fecha de la ultima actualizacion de avisos
Set rst1 = dbs.OpenRecordset("SELECT * FROM ActualizacionAvisos")
If Not rst1.EOF Then
   rst1.MoveFirst
   'Si la fecha es la de hoy los avisos están actualizados y nos salimos sin hacer nada
   If rst1!FechaActualizacion = Date Then
      rst1.Close
      Set rst1 = Nothing
      Set dbs = Nothing
      Exit Function
   End If
End If

   'Si no están actualizados los avisos (solo si la FechaActualizacion es menor que la de hoy
   'ejecutamos el procedimiento para actualizar los avisos
   Set rst = dbs.OpenRecordset("Select * from Avisos") 'abrimos un recordset de la tabla avisos
```

```
37
38    If Not rst.EOF Then
39      rst.MoveLast
40      rst.MoveFirst
41      For i = 1 To rst.RecordCount
42        If rst!Periodico = True Then 'Si el aviso es periódico
43          Dim periodo 'declaramos una variable para guardar el tipo de periodicidad
44          Select Case rst!Periodicidad 'averiguamos el tipo de periodicidad
45            Case 1
46              periodo = "d"
47            Case 2
48              periodo = "ww"
49            Case 3
50              periodo = "m"
51            Case 4
52              periodo = "yyyy"
53          End Select
54          If rst!ProximoAviso < Date Then 'Si proximo aviso es menor de hoy
55            If rst!Periodicidad = 1 Then 'Si periodicidad es 1 (=diaria)
56              rst.Edit
57              'sumamos a la fecha actual de ProximoAviso el numero de periodos establecido
58              'en NumeroPeriodos y del tipo que antes hemos seleccionado en la variable periodo
59              rst!ProximoAviso = DateAdd(periodo, rst!NumeroPeriodos, rst!ProximoAviso)
60              rst!NumeroAviso = 3 'Si aviso es diario no se anticipa, así que el numero de aviso sie
61              rst.Update
62            Else 'para los demas tipos de periodicidad solamente tenemos que realizar la suma
63              rst.Edit
64              rst!ProximoAviso = DateAdd(periodo, rst!NumeroPeriodos, rst!ProximoAviso)
65              rst.Update
66            End If
67          End If
68        End If
69
70        'Si está seleccionado anticipar aviso actualizamos tambien las fechas de los dos avisos
71        'anticipados (3 y 6 dias antes de la fecha del acontecimiento)
72        If rst!AnticiparAviso = True Then
73          'Si no estamos en el periodo comprendido entre el dia del aviso y 6 dias antes
74          'no corresponde avisar y, el proximo aviso será el primero de la serie
75          If Date < DateAdd("d", -7, rst!ProximoAviso) Or Date > rst!ProximoAviso Then
76            rst.Edit
77            rst!Avisar = False
78            rst!NumeroAviso = 1
79            rst.Update
80          'Si estamos en una fecha comprendida entre los 6 y los 4 dias anteriores al aviso
```

```
        ElseIf Date > DateAdd("d", -7, rst!ProximoAviso) And Date <= DateAdd("d", -4, rst!ProximoAviso) Then
          'Si numero aviso es 1 es que todavía no se ha avisado y ponemos avisar a true
          'para que se produzca el aviso
          If rst!NumeroAviso = 1 Then
            rst.Edit
            rst!Avisar = True
            rst.Update
          'Si avisar es distinto de 1 es que ya se ha producido el primer aviso y por tanto
          'ponemos avisar a false hasta que llegue la fecha del segundo aviso
          Else
            rst.Edit
            rst!Avisar = False
            rst.Update
          End If
          'Si estamos en una fecha comprendida entre 4 antes y el dia antes del aviso
        ElseIf Date > DateAdd("d", -4, rst!ProximoAviso) And Date < rst!ProximoAviso Then
          'Si numeroaviso es menor que 3 hay que dar un aviso, así que ponemos avisar a true
          If rst!NumeroAviso < 3 Then
            rst.Edit
            rst!Avisar = True
            rst.Update
          Else 'Si NumeroAviso es igual a 3 es que ya se han producido los dos primeros avisos
            rst.Edit
            rst!Avisar = False
            rst.Update
          End If
          'Si hoy es la fecha del aviso
        ElseIf Date = rst!ProximoAviso Then
          If rst!NumeroAviso < 4 Then 'Mientras NumeroAviso sea menor que 4 toca avisar
            rst.Edit
            rst!Avisar = True
            rst.Update
          Else
            rst.Edit
            rst!Avisar = False
            rst.Update
          End If
        End If
      'Si no está maracado AnticiparAviso solo hay que avisar el dia de ProximoAviso
      ElseIf rst!AnticiparAviso = False Then
        'Si hoy es el dia del ProximoAviso y NumeroAviso es igual a 3 aun no se ha avisado
        If Date = rst!ProximoAviso And rst!NumeroAviso = 3 Then
          rst.Edit
          rst!Avisar = True
```

```
            rst.Update
          'Si es distinto de 3 es que ya se ha avisado y ponemos avisar a False
          ElseIf Date = rst!ProximoAviso And rst!NumeroAviso <> 3 Then
            rst.Edit
            rst!Avisar = False
            rst.Update
          Else 'Si hoy no es el dia del proximo aviso ponemos Avisar a False y dejamos NumeroAvis para la proxima ocasion
            rst.Edit
            rst!Avisar = False
            rst!NumeroAviso = 3
            rst.Update
          End If
        End If
        rst.MoveNext
      Next i
    End If

  rst1.Edit
  'Cambiamos la fecha de Actualización a Hoy para que sepamos que ya están actualizados los aviso de hoy
  rst1!FechaActualizacion = Date
  rst1.Update

  rst1.Close
  rst.Close
  Set rst1 = Nothing
  Set rst = Nothing
  Set dbs = Nothing

End Function

'*****************************************************************************
'* CompruebaAvisos
'* Comprueba si toca emitir algún mensaje de aviso. Para ello mira si en la tabla
'* Avisos existe algun registro con el campo Booleano Avisar a True
'* Argumentos:
'* uso: CompruebaAvisos

'*****************************************************************************

Function CompruebaAvisos() As Boolean
Dim dbs As DAO.Database, rst As DAO.Recordset
Dim i As Integer

```

```
CompruebaAvisos = False 'Inicializamos a false

Set dbs = CurrentDb
'Abrimos un recordset de la tabla Avisos y lo recorremos hasta encontrar un registro
'cuyo campo Avisar este a true. Si lo encontramos devolvemos valor True, en otro caso False
Set rst = dbs.OpenRecordset("Select * from Avisos")

If Not rst.EOF Then
   rst.MoveLast
   rst.MoveFirst
   For i = 1 To rst.RecordCount
      If rst!Avisar = True Then
         CompruebaAvisos = True
         Exit For
      End If
      rst.MoveNext
   Next i
End If

rst.Close
Set rst = Nothing
Set dbs = Nothing
End Function

'*********************************************************************************
'* EmiteAvisos
'* Recorre la Tabla Avisos buscando los registros que tengan marcado Avisar a True
'* y emite el correspondiente aviso
'* Argumentos:
'* uso: EmiteAvisos

'*********************************************************************************

Function EmiteAvisos()
Dim dbs As DAO.Database, rst As DAO.Recordset
Dim fechaMensaje
Dim i As Integer

Set dbs = CurrentDb
'Abrimos un recordset de la tabla Avisos con aquellos registros que tienen Avisar a True
Set rst = dbs.OpenRecordset("Select * from Avisos WHERE Avisar= True")
If Not rst.EOF Then
   rst.MoveLast
   rst.MoveFirst
```

```
212     'Hacemos un bucle para recorrer el conjunto de registros devuelto
213     For i = 1 To rst.RecordCount
214       'Si aviso es 0 es un error y no corresponde avisar
215       If rst!Avisar = True And rst!NumeroAviso > 0 Then
216         'Con este if seleccionamos si el texo dirá "El proximo...." u "Hoy.."
217         'y construimos la cadena con la fecha de aviso
218         If rst!ProximoAviso > Date Then
219           fechaMensaje = "el próximo" & Chr(13) & Chr(13) & Format(rst!ProximoAviso, "dddd",
vbMonday) & " " & rst!ProximoAviso
220         Else
221           fechaMensaje = "hoy" & Chr(13) & Chr(13) & Format(rst!ProximoAviso, "dddd", vbMonc
" & rst!ProximoAviso
222         End If
223         Beep 'Emitimos un beep sonoro y mostramos el mensaje
224         MsgBox "Vd. Solicito que se le recordara que " & fechaMensaje & Chr(13) & Chr(13) & _
225             UCase(rst!MensajeAviso), vbInformation + vbOKOnly, "AVISOS: RECORDATORIO"
226         rst.Edit
227         'Una vez emitido el aviso Actualizamos NumeroAviso al proximo número
228         If rst!NumeroAviso = 3 Then 'Si ya era el tercero, ya no hay mas avisos, lo ponemos a 0
229           rst!NumeroAviso = 0
230         Else 'Si es menor de 3, le sumamos 1 para que quede preparado para el proximo
231           rst!NumeroAviso = rst!NumeroAviso + 1
232         End If
233         rst!Avisar = False 'Ponemos avisar a false, hasta que la Funcion ActualizaAvisos lo cambie cu
corresponda
234         rst.Update
235       End If
236       rst.MoveNext
237     Next i
238   End If
239   rst.Close
240   Set rst = Nothing
241   Set dbs = Nothing
242
243   End Function
244
245   '*****************************************************************************
246   '* DecodeDiaSemana
247   '* Decodifica el Valor numérico de dia de la semana
248   '* Argumentos:
249   '* uso: DecodeDiaSemana
250
251   '*****************************************************************************
252
253   Function DecodeDiaSemana() As String
254   Dim frm As Form
```

```
Set frm = Screen.ActiveForm
Select Case frm!DiaSemana
   Case 1
      DecodeDiaSemana = "Lunes"
   Case 2
      DecodeDiaSemana = "Martes"
   Case 3
      DecodeDiaSemana = "Miercoles"
   Case 4
      DecodeDiaSemana = "Jueves"
   Case 5
      DecodeDiaSemana = "Viernes"
   Case 6
      DecodeDiaSemana = "Sabado"
   Case 7
      DecodeDiaSemana = "Domingo"
End Select
Set frm = Nothing
End Function


'*********************************************************************************
'* DecodePeriodicidad
'* Decodifica el valor numérico de tipo de periodicidad
'* Argumentos:
'* uso: DecodePeriodicidad

'*********************************************************************************

Function DecodePeriodicidad() As String
Dim frm As Form
Set frm = Screen.ActiveForm
Select Case frm!Periodicidad
   Case 1
      DecodePeriodicidad = "Dias"
   Case 2
      DecodePeriodicidad = "Semanas"
   Case 3
      DecodePeriodicidad = "Meses"
   Case 4
      DecodePeriodicidad = "Años"
End Select
Set frm = Nothing
End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	20/4/2016 13:40:38
LastUpdated:	20/4/2016 13:40:38	Owner:	admin
UserName:	admin		

Código

```
1  Attribute VB_Name = "MdlBarrasBotones"
2  Option Compare Database
3  Option Explicit
4
5
6  'Va al primer registro
7  Function PrimerRegistro()
8  On Error GoTo Err_PrimerRegistro
9
10 DoCmd.GoToRecord , , acFirst
11
12 Exit_PrimerRegistro:
13 Exit Function
14 Err_PrimerRegistro:
15 MsgBox Err.Description
16 Resume Exit_PrimerRegistro
17 End Function
18
19 'Va al ultimo Registro
20 Function UltimoRegistro()
21 On Error GoTo Err_UltimoRegistro
22
23 DoCmd.GoToRecord , , acLast
24
25 Exit_UltimoRegistro:
26 Exit Function
27 Err_UltimoRegistro:
28 MsgBox Err.Description
29 Resume Exit_UltimoRegistro
30 End Function
31
32 'Va al siguiente registro
33 Function RegSiguiente()
34 On Error GoTo Err_RegSiguiente
35
36 DoCmd.GoToRecord , , acNext
```

```
Exit_RegSiguiente:
Exit Function
Err_RegSiguiente:
MsgBox Err.Description
Resume Exit_RegSiguiente
End Function

'Va al registro anterior
Function RegAnterior()
On Error GoTo Err_RegAnterior

DoCmd.GoToRecord , , acPrevious

Exit_RegAnterior:
Exit Function
Err_RegAnterior:
MsgBox Err.Description
Resume Exit_RegAnterior
End Function

'Agrega un registro
Function AgregaRegistro()
On Error GoTo Err_AgregaRegistro

DoCmd.GoToRecord , , acNewRec

Exit_AgregaRegistro:
Exit Function
Err_AgregaRegistro:
MsgBox Err.Description
Resume Exit_AgregaRegistro
End Function

Function EliminaRegistro()
On Error GoTo EliminaRegistro_Err
On Error Resume Next

DoCmd.GoToControl Screen.PreviousControl.NAME
Err.Clear

If (Not frm.NewRecord) Then
    DoCmd.RunCommand acCmdDeleteRecord
End If
```

```
81
82  If (frm.NewRecord And Not frm.Dirty) Then
83      Beep
84  End If
85
86  If (frm.NewRecord And frm.Dirty) Then
87      DoCmd.RunCommand acCmdUndo
88  End If
89
90  If (MacroError <> 0) Then
91      Beep
92      MsgBox MacroError.Description, vbOKOnly, ""
93  End If
94
95  EliminaRegistro_Exit:
96  Exit Function
97  EliminaRegistro_Err:
98  MsgBox Error$
99  Resume EliminaRegistro_Exit
100 End Function  'EliminaRegistro()
101
102 'Busca un registro
103 Function BuscaRegistro()
104 On Error GoTo Err_BuscaRegistro
105
106 Screen.PreviousControl.SetFocus
107 DoCmd.DoMenuItem acFormBar, acEditMenu, 10, , acMenuVer70
108
109 Exit_BuscaRegistro:
110 Exit Function
111 Err_BuscaRegistro:
112 MsgBox Err.Description
113 Resume Exit_BuscaRegistro
114 End Function
115
116 Function BuscaRapido()
117 On Error GoTo BuscaRapido_Err
118
119 With CodeContextObject
120     On Error Resume Next
121     DoCmd.GoToControl Screen.PreviousControl.NAME
122     Err.Clear
123     DoCmd.RunCommand acCmdFind
124     If (.MacroError <> 0) Then
```

```
125                 Beep
126                 MsgBox .MacroError.Description, vbOKOnly, ""
127         End If
128 End With
129
130 BuscaRapido_Exit:
131 Exit Function
132 BuscaRapido_Err:
133 MsgBox Error$
134 Resume BuscaRapido_Exit
135 End Function
136
137 'Codigo generado en parte por el Command Button Wizard
138 Function VistaBD() 'Botón Vista_BD
139 Dim TxtNombreDoc As String
140 On Error GoTo Err_VistaBD
141
142 TxtNombreDoc = "Clientes"
143 'Cerrar el formulario Panel de control principal.
144 DoCmd.Close
145 'Mandar el enfoque a la ventana de base de datos. Selecciona la tabla "........."
146 DoCmd.SelectObject acTable, TxtNombreDoc, True
147
148 Salir_VistaBD:
149 Exit Function
150 Err_VistaBD:
151 MsgBox Err.Description
152 Resume Salir_VistaBD
153 End Function
154
155 Public Function BloqueaBotones(NombreFormulario As Form)
156 'Deshabilito ó habilito botones de navegación según el registro donde estemos
157 If NombreFormulario.NRec = 1 Then
158         NombreFormulario.PrimerRegistro.Enabled = False 'Primer registro
159         NombreFormulario.RegAnterior.Enabled = False 'Anterior
160 Else
161         NombreFormulario.PrimerRegistro.Enabled = True 'Primer registro
162         NombreFormulario.RegAnterior.Enabled = True 'Anterior
163 End If
164
165 If NombreFormulario.NRec = NombreFormulario.Rec Then
166         NombreFormulario.RegSiguiente.Enabled = False 'Siguiente
167         NombreFormulario.UltimoRegistro.Enabled = False 'Ultimo
168         'MsgBox "Estamos en el último Registro.!!" & (Chr(13)) & (Chr(10)) & "Selecciona Anterior, P
    ó elige nuevas condiciones de Filtrado", vbInformation, "INFORMACION PARA USUARIO"
```

```
Else
    NombreFormulario.RegSiguiente.Enabled = True 'Siguiente
    NombreFormulario.UltimoRegistro.Enabled = True 'Ultimo
End If

End Function

Function BotoneraRegistros()
On Error GoTo BotoneraRegistros_TratamientoErrores

If frm.RecordsetClone.RecordCount = 0 Then
        'Si no se devuelve ningún registro bloqueamos los botones de desplazamiento y mostramos mensaje.
        frm.PrimerRegistro.Enabled = False 'Primer registro
        frm.RegAnterior.Enabled = False 'Anterior
        frm.RegSiguiente.Enabled = False 'Siguiente
        frm.UltimoRegistro.Enabled = False 'Ultimo
        frm.NRec = 0
        frm.Rec = 0
        Beep
        MsgBox "No hay datos especificados para esta Selección", vbInformation, "INFORMACION USUARIO"
        Set frm = Nothing
        Exit Function
End If

'Lineas de código para el visor de registros---
frm!NRec = frm.CurrentRecord
frm.RecordsetClone.MoveLast
frm!Rec = frm.RecordsetClone.RecordCount 'Cuenta los registros del Origen del Registro

'Deshabilito ó habilito los Botones según el registro en el que estamos
If frm!NRec = 1 Then
    frm!PrimerRegistro.Enabled = False 'Primer registro
    frm!RegAnterior.Enabled = False 'Anterior
Else
    frm!PrimerRegistro.Enabled = True 'Primer registro
    frm!RegAnterior.Enabled = True 'Anterior
End If

If frm!NRec = frm!Rec Then
    frm!RegSiguiente.Enabled = False 'Siguiente
    frm!UltimoRegistro.Enabled = False 'Ultimo
    'MsgBox "Estamos en el último Registro.!!" & (Chr(13)) & (Chr(10)) & "Pulsa Registro Anterio Primero para seguir visualizando", vbInformation, "INFORMACION PARA USUARIO"
Else
```

```
212         frm!RegSiguiente.Enabled = True 'Siguiente
213         frm!UltimoRegistro.Enabled = True 'Ultimo
214     End If
215
216     BotoneraRegistros_Salir:
217     On Error GoTo 0
218     Exit Function
219     BotoneraRegistros_TratamientoErrores:
220     MsgBox "Error " & Err.Number & " en Funcion.: BotoneraRegistros de Modulo: BarrasBotones (" &
        Err.Description & ")", vbCritical + vbOKOnly, "ATENCION"
221     Resume BotoneraRegistros_Salir
222     End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	3/2/2015 21:03:52
LastUpdated:	3/2/2015 21:03:52	Owner:	admin
UserName:	admin		

Código

```
1  Attribute VB_Name = "mdlCalendario1"
2  Option Compare Database
3  Option Explicit
4  Public fechaDevuelta As Date
5  Function Calendario()
6    DoCmd.OpenForm "frmCalendar", , , , , acDialog
7  End Function
8
9  Function resultCalendario()
10 fechaDevuelta = Format(Forms!frmCalendar!lblResultado.Caption, "dd/mm/yyyy")
11 DoCmd.Close acForm, "frmCalendar"
12 End Function
13
14
15
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	20/4/2016 13:40:38
Description:	Funciones para mostrar y quitar el cursor de la mano	LastUpdated:	20/4/2016 13:40:38
Owner:	admin	UserName:	admin

Código

```
Attribute VB_Name = "MdlHandCursor"
Option Compare Database
Option Explicit

'Módulo: MdlHandCursor >> Fecha : 25/11/2007 >> Autor : Patxi Sanz >>  Propósito : Cambiar el
activo por la manita.
'Nota     : Para volver a establecer el cursor por defecto, tan sólo hay que usar el objeto Screen y su
propiedad MousePointer:
'Screen.MousePointer = 0
'Méritos: http://blogs.msdn.com/clintcovington/archive/2006/12/22/interactive-chart-filtering-co
sample.aspx

Private Const HandCursor = 32649&
Private Const HandCursor1 = 32648&



'Function  : SetHandCursor.  Propósito : Establece el cursor actual a la mano. Parámetros: Ninguno
'Devuelve  : El Id del Cursor que se muestra, o 0 si se ha fallado a la llamada.
Public Function SetHandCursor() As Long
SetHandCursor = SetCursor(LoadCursor(0, HandCursor))
End Function

Public Function SetHandCursor1() As Long
SetHandCursor1 = SetCursor(LoadCursor(0, HandCursor1))
End Function

'Function  : SetDefaultCursor.  Propósito : Establecer el cursor por defecto.  Parámetros: Ninguno.
Devuelve  : Nada.
Public Function SetDefaultCursor()
   Screen.MousePointer = 0
End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	20/4/2016 13:40:38
LastUpdated:	20/4/2016 13:40:38	Owner:	admin
UserName:	admin		

Código

```
Attribute VB_Name = "MdlPrinters"
Option Compare Database
Option Explicit

'GetDefaultPrinter: (Access MVP)

Function GetDefaultPrinter() As String
Dim StrDefault As String
Dim lngbuf As Long

StrDefault = String(255, Chr(0))
lngbuf = GetProfileString("Windows", "Device", "", StrDefault, Len(StrDefault))
If lngbuf > 0 Then
    GetDefaultPrinter = Left$(StrDefault, InStr(StrDefault, ",") - 1)
Else
    GetDefaultPrinter = ""
End If

End Function

'SetDefaultPrinter:

Function SetDefaultPrinter(PrinterName As String) As Boolean
Dim WshNetwork As Object 'New wshNetwork
On Error GoTo Err_SetDefaultPrinter
Set WshNetwork = CreateObject("WScript.Network")
WshNetwork.SetDefaultPrinter PrinterName
Set WshNetwork = Nothing
SetDefaultPrinter = True

Err_SetDefaultPrinter:

End Function

'Printers

```

```
Function Printers() As Collection
Dim WshNetwork As Object 'New wshNetwork
Dim Col As New Collection
Dim i As Integer

Set WshNetwork = CreateObject("WScript.Network")

With WshNetwork.EnumPrinterConnections
    For i = 0 To .Count - 1
        If i Mod 2 <> 0 Then
            Col.Add .Item(i)
        End If
    Next
End With

Set Printers = Col

Set Col = Nothing
Set WshNetwork = Nothing

End Function

'OpenReportPrt

Sub OpenReportPrt(reportName, Optional View = acViewNormal, Optional FilterName, Optional
WhereCondition, Optional Printer)
Dim DefaultPrt As String

'Guardamos el valor de la impresora predeterminada
DefaultPrt = GetDefaultPrinter

'Hacemos predeterminada la impresora que hemos escogido
SetDefaultPrinter Nz(Printer)

'Imprimimos el informe con los parametros pasados a la rutina
On Error Resume Next
DoCmd.OpenReport reportName, View, FilterName, WhereCondition

'Ponemos como predeterminada la impresora que estaba
SetDefaultPrinter DefaultPrt

End Sub

Public Sub CreaPDF()
Dim RutaPDF As String
```

```
81  Dim NombreInfPDF As String
82  Dim RutaYFicheroPDF As String
83  Dim NumFicheros As Integer
84  On Error GoTo CreaPdF_TratamientoErrores
85
86
87  'Defino la Ruta donde están los Ficheros PDF
88  RutaPDF = Application.CurrentProject.Path & "\InformesPDF\"
89
90  'Para evitar Error si no hay Ficheros
91  NumFicheros = CreateObject("Scripting.FileSystemObject").GetFolder(RutaPDF).Files.Count
92  If NumFicheros > 0 Then
93      'Borro todos los Ficheros que haya en el Directorio
94      Kill Application.CurrentProject.Path & "\InformesPDF\*.*"
95  End If
96
97  NombreInfPDF = NombreInforme & ".pdf"
98  RutaYFicheroPDF = RutaPDF & NombreInfPDF
99
100 'Aquí exportamos y visualizamos el Informe en PDF. Para visualizar >> AutoStart: = True
101 DoCmd.OutputTo ObjectType:=acOutputReport, ObjectName:=NombreInforme,
    OutputFormat:=acFormatPDF, OutputFile:=RutaYFicheroPDF, AutoStart:=False 'True
102
103
104
105 CreaPdF_Salir:
106 On Error GoTo 0
107 Exit Sub
108 CreaPdF_TratamientoErrores:
109 MsgBox "Error " & Err.Number & " en Procedimiento.: CreaPdF de Documento VBA: MdlPrinters (
    Err.Description & ")"
110 Resume CreaPdF_Salir
111 End Sub
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	20/4/2016 13:40:38
LastUpdated:	20/4/2016 13:40:38	Owner:	admin
UserName:	admin		

Código

```
1  Attribute VB_Name = "MdlVariables"
2  Option Compare Database
3  Option Explicit
4
5
6  Public frm As Form
7  Public Rpt As Report
8
9  Public RutaApp As String
10
11 Public CriterioUno As String
12 Public CriterioDos As String
13 Public CriterioTres As String
14 Public CriterioCuatro As String
15 Public CriterioCinco As String
16
17 Public Criterios As String
18
19 Public dbs As DAO.Database
20 Public strSQL As String
21 Public rst As DAO.Recordset
22
23 Public NombreInforme As String
24
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	8/12/2015 14:58:19
LastUpdated:	8/12/2015 14:58:19	Owner:	admin
UserName:	admin		

Código

```
Attribute VB_Name = "modScaleForm"
''''''
```

```
37  '           .) Thanks to Myke Mayers for his function AdjustColumnWidths, it helped me a lot!
38  ' Requires   : :) This module needs the class module clFormWindow:
39  '              Visit http://www.mvps.org/access/forms/frm0042.htm
40  '              and download http://www.mvps.org/access/downloads/clFormWindow.bas
41  ' Remarks    : .) I am pretty sure that there are many errors in this module. Feel free
42  '              to correct them yourself or simply inform me about them.
43  '           .) This module is provided "as is", I do not take responsability for usage.
44  '           .) This module is licensed under the GPL.
45  '           .) Thanks to all other Access developers posting solutions and code helping me
46  '              over the last years!
47  ' Remarks 2  : .) Resizing (sub)forms with many controls takes long (I know of a case, where it
48  '              takes 15 seconds).
49  '           .) Fonts are sized using the smaller value of either horizontal or vertical scale
50  '              factor - if this would not be the case, fonts would not fit any more into
51  '              their controls.
52  '           .) Windows are centered after rescaling, so that they can not "disappear" to
53  '              non-visible areas. Therefore, I use the class module "clFormWindow". If you only
54  '              want the module to set the new size but not center the form, set the optional
55  '              parameter "DoNotCenterForm" to "true".
56  '''''''
```

```
81  ' 2008-01-21: .) Fixed a bug at scaling column widths
82  '
83  ' 2008-01-18: .) First control element returned from Access has been been included in array (missi
84  '              "ReDim Preserve" added)
85  '
86  ' 2008-01-07: .) DPI changes are reflected in scale factors
87  '            .) Added function "InitialInsertCodeIntoForms"
88  '
89  ' 2007-12-01: .) Split ModScaleForm in several functions
90  '
91  ' 2007-11-01: .) First thoughts and research
92  '            .) Decision to create modScaleForm
93  ''''''
```

```
    DPI As Long
End Type


Private Const DefaultDesignWidth As Long = 1400
Private Const DefaultDesignHeight As Long = 1050
Private Const DefaultDesignDPI As Long = 96
Private Const WM_HORZRES As Long = 8
Private Const WM_VERTRES As Long = 10
Private Const WM_LOGPIXELSX As Long = 88



Public arrCtlsScaleForm() As tControl 'public array for control properties, available for all forms

Public Sub ScaleFormWindow( _
    ByVal frm As Access.Form, _
    Optional DoNotScaleFormProportionally As Boolean, _
    Optional DoNotCenterForm As Boolean)

Dim CurrentFormHeight As Long
Dim CurrentFormName As String
Dim CurrentFormWidth As Long
Dim CurrentScreenDPI As Long
Dim CurrentScreenHeight As Long
Dim CurrentScreenWidth As Long
Dim DesignResDPI As Long
Dim DesignResHeight As Long
Dim DesignResWidth As Long
Dim GetScreenResolution As tDisplay
Dim hDCcaps As Long
Dim lngRtn As Long
Dim MonitorResHeightRatio As Single
Dim MonitorResWidthRatio As Single
Dim NewForm As New clFormWindow
Dim NewFormHeight As Long
Dim NewFormWidth As Long
Dim OnOpenWindowHeight As Long
Dim OnOpenWindowWidth As Long
Dim rectWindow As tRect
Dim ScaledFormString As String
Dim TagString As String
Dim TagStringArray() As String
Dim TagStringPosition As Long
```

```
169
170
171 On Error Resume Next
172
173 CurrentFormName = frm.NAME
174
175 'Call API to get current resolution
176 hDCcaps = WM_apiGetDC(0) 'Get display context for desktop (hwnd = 0).
177 With GetScreenResolution
178    .Height = WM_apiGetDeviceCaps(hDCcaps, WM_VERTRES)
179    .Width = WM_apiGetDeviceCaps(hDCcaps, WM_HORZRES)
180    .DPI = WM_apiGetDeviceCaps(hDCcaps, WM_LOGPIXELSX)
181 End With
182 lngRtn = WM_apiReleaseDC(0, hDCcaps) 'Release display context.
183
184 CurrentScreenWidth = GetScreenResolution.Width
185 CurrentScreenHeight = GetScreenResolution.Height
186 CurrentScreenDPI = GetScreenResolution.DPI
187
188 ScaledFormString = ("_mSF" & CurrentScreenWidth & "x" & CurrentScreenHeight & "x" &
    CurrentScreenDPI)
189
190 If isSubform(frm) Then
191    Exit Sub
192 Else
193    If Right(CurrentFormName, Len(ScaledFormString)) = ScaledFormString Then
194       Exit Sub 'this form is statically scaled for the current resolution
195    Else
196    'go go go
197    End If
198 End If
199
200
201 'Set size to fit form
202 DoCmd.RunCommand acCmdSizeToFitForm
203
204
205 'Extract "DesignRes:*:" from Tag-Option of Form
206 TagString = LCase(frm.Tag) 'lower case tag-field
207 TagStringPosition = InStr(TagString, "designres:") 'get left position of resolution
208 If TagStringPosition > 0 Then
209    TagString = Mid(TagString, TagStringPosition + 10) 'get string starting with resolution (the "+1
    char count of "DesignRes:"
210    TagStringPosition = InStr(TagString, ":") 'Get Position of ":" behind resolution
211    TagString = Left(TagString, TagStringPosition - 1) 'remove everything behind ":" - the variable
    contains the design resolution
```

```
212     ReDim TagStringArray(1)
213     TagStringArray = Split(TagString, "x") '"x" is the delimiter
214     If UBound(TagStringArray) >= 0 Then DesignResWidth = TagStringArray(0)
215     If UBound(TagStringArray) >= 1 Then DesignResHeight = TagStringArray(1)
216     If UBound(TagStringArray) >= 2 Then DesignResDPI = TagStringArray(2)
217     If (DesignResWidth <= 0) Or (DesignResHeight <= 0) Or (DesignResDPI <= 0) Then
218        DesignResWidth = DefaultDesignWidth
219        DesignResHeight = DefaultDesignHeight
220        DesignResDPI = DefaultDesignDPI
221     End If
222     Erase TagStringArray
223  Else
224     'DesignRes is not specified in the form so we use default values for scaling
225     DesignResWidth = DefaultDesignWidth
226     DesignResHeight = DefaultDesignHeight
227     DesignResDPI = DefaultDesignDPI
228  End If
229
230
231  'Extract "OnOpenRes:*:" from Tag-Option of Form
232  TagString = LCase(frm.Tag) 'lower case tag-field
233  TagStringPosition = InStr(TagString, "onopenres:") 'get left position of resolution
234  If TagStringPosition > 0 Then
235     TagString = Mid(TagString, TagStringPosition + 10) 'get string starting with resolution (the "+1
     char count of "OnOpenRes:"
236     TagStringPosition = InStr(TagString, ":") 'Get Position of ":" behind resolution
237     TagString = Left(TagString, TagStringPosition - 1) 'remove everything behind ":" - the variable
     contains the OnOpen resolution
238     ReDim TagStringArray(1)
239     TagStringArray = Split(TagString, "x") '"x" is the delimiter
240     OnOpenWindowWidth = TagStringArray(0)
241     OnOpenWindowHeight = TagStringArray(1)
242     Erase TagStringArray
243  Else
244     Call WM_apiGetWindowRect(frm.hWnd, rectWindow) 'prepare for getting size of current windo
     screen
245     OnOpenWindowWidth = rectWindow.Right - rectWindow.Left
246     OnOpenWindowHeight = rectWindow.Bottom - rectWindow.Top
247     frm.Tag = frm.Tag & ", OnOpenRes:" & OnOpenWindowWidth & "x" & OnOpenWindowHeight
248  End If
249
250
251
252  If (DesignResDPI = CurrentScreenDPI) And (DesignResWidth = CurrentScreenWidth) And
     (DesignResHeight = CurrentScreenHeight) Then
253     'design values match current values, no need for scaling
```

```
254     Exit Sub
255 Else
256     'design values and current values are not equal - go on
257 End If
258
259
260 'Calculate Ratios
261 MonitorResHeightRatio = (CurrentScreenHeight / DesignResHeight) * (DesignResDPI / CurrentScr
262 MonitorResWidthRatio = (CurrentScreenWidth / DesignResWidth) * (DesignResDPI / CurrentScree
263
264
265 'If DoNotScaleFormProportionally is True, then the ratios remain the same. Else, the smaller value
    for both
266 If DoNotScaleFormProportionally = False Then
267     If MonitorResHeightRatio < MonitorResWidthRatio Then MonitorResWidthRatio =
    MonitorResHeightRatio
268     If MonitorResWidthRatio < MonitorResHeightRatio Then MonitorResHeightRatio =
    MonitorResWidthRatio
269 Else
270     'leave ratios as they are and scale unproportionally, strictly as display resolution ratios are.
271 End If
272
273
274 'Get width and heights of form
275 Call WM_apiGetWindowRect(frm.hWnd, rectWindow) 'prepare for getting size of current window
276 CurrentFormWidth = rectWindow.Right - rectWindow.Left
277 CurrentFormHeight = rectWindow.Bottom - rectWindow.Top
278
279
280 'Calculate new width and height (add 1 pixel for safety reasons), including percentage relative to v
    desktop
281 NewFormHeight = Round(CurrentFormHeight * MonitorResHeightRatio, 0) '+ 1
282 NewFormWidth = Round(CurrentFormWidth * MonitorResWidthRatio, 0) '+ 1
283
284
285 'Set form's new size and position
286 NewForm.hWnd = frm.hWnd
287 With NewForm
288     .Height = NewFormHeight
289     .Width = NewFormWidth
290 End With
291 'If DoNotCenterForm is True, then the form is not centered
292 If DoNotCenterForm = False Then
293     With NewForm
294         .Top = (.Parent.Height - NewFormHeight) / 2
295         .Left = (.Parent.Width - NewFormWidth) / 2
```

```
    End With
Else
    'form should not be centered
End If
Set NewForm = Nothing


End Sub

Public Sub ScaleFormControls( _
    ByVal frm As Access.Form, _
    Optional NoScalingWhenSubform As Boolean)

Dim ArrayCount As Long
Dim ctl As Control
Dim CurrentWindowHeight As Long
Dim CurrentWindowWidth As Long
Dim FormExists As Boolean
Dim LastScaleHorzFactor As Single
Dim LastScaleVertFactor As Single
Dim OnOpenWindowHeight As Long
Dim OnOpenWindowWidth As Long
Dim rectWindow As tRect
Dim ScaleFontFactor As Single
Dim ScaleHorzFactor As Single
Dim ScaleVertFactor As Single
Dim TagString As String
Dim TagStringArray() As String
Dim TagStringPosition As Long


On Error Resume Next 'no error messages, no debugger will start

'prepare array holding initial control sizes
If (Not arrCtlsScaleForm) = -1 Then
    'array empty, so create one
    ReDim arrCtlsScaleForm(1) 'create array
Else
    'array exists, do nothing
End If


If isSubform(frm) And (NoScalingWhenSubform = True) Then
    Exit Sub
```

```
Else
    'go go go
End If


'Extract "OnOpenRes:*:" from Tag-Option of Form
TagString = LCase(frm.Tag) 'lower case tag-field
TagStringPosition = InStr(TagString, "onopenres:") 'get left position of resolution
If TagStringPosition > 0 Then
    TagString = Mid(TagString, TagStringPosition + 10) 'get string starting with resolution (the "+1
char count of "OnOpenRes:"
    TagStringPosition = InStr(TagString, ":") 'Get Position of ":" behind resolution
    TagString = Left(TagString, TagStringPosition - 1) 'remove everything behind ":" - the variable
contains the OnOpen resolution
    ReDim TagStringArray(1)
    TagStringArray = Split(TagString, "x") '"x" is the delimiter
    OnOpenWindowWidth = TagStringArray(0)
    OnOpenWindowHeight = TagStringArray(1)
    Erase TagStringArray
Else
    Call WM_apiGetWindowRect(frm.hWnd, rectWindow) 'prepare for getting size of current windo
screen
    OnOpenWindowWidth = rectWindow.Right - rectWindow.Left
    OnOpenWindowHeight = rectWindow.Bottom - rectWindow.Top
    frm.Tag = frm.Tag & ", OnOpenRes:" & OnOpenWindowWidth & "x" & OnOpenWindowHeight
End If


'check if form has already been opened and is part of the array
'if not, write the properties to the array
FormExists = 0 'means name of form is not in the array, initially
For ArrayCount = LBound(arrCtlsScaleForm) To UBound(arrCtlsScaleForm)
    If arrCtlsScaleForm(ArrayCount).FormName = frm.NAME Then FormExists = 1 'form name is alr
the array, no need to write array
Next ArrayCount


If FormExists = 0 Then
    ArrayCount = UBound(arrCtlsScaleForm) + 1
    ReDim Preserve arrCtlsScaleForm(ArrayCount) 'Increase the size of the array.
    For Each ctl In frm.Controls
        With arrCtlsScaleForm(ArrayCount)
            .FormName = frm.NAME
            .NAME = ctl.NAME
            .Height = ctl.Height
            .Width = ctl.Width
```

```
        .Top = ctl.Top
        .Left = ctl.Left
        .FontSize = ctl.FontSize
        .ColumnWidths = ctl.ColumnWidths
        .ListWidth = ctl.ListWidth
        .TabFixedWidth = ctl.TabFixedWidth
        .TabFixedHeight = ctl.TabFixedHeight
      End With
      ArrayCount = ArrayCount + 1
      ReDim Preserve arrCtlsScaleForm(ArrayCount) 'Increase the size of the array.
    Next ctl
    'save initial heights for header, detail, footer
    ArrayCount = UBound(arrCtlsScaleForm) + 1
    ReDim Preserve arrCtlsScaleForm(ArrayCount)
    With arrCtlsScaleForm(ArrayCount)
      .FormName = frm.NAME
      .NAME = "xxxHeaderxxx"
      .Height = frm.Section(Access.acHeader).Height
    End With
    ArrayCount = ArrayCount + 1
    ReDim Preserve arrCtlsScaleForm(ArrayCount)
    With arrCtlsScaleForm(ArrayCount)
      .FormName = frm.NAME
      .NAME = "xxxDetailxxx"
      .Height = frm.Section(Access.acDetail).Height
    End With
    ArrayCount = ArrayCount + 1
    ReDim Preserve arrCtlsScaleForm(ArrayCount)
    With arrCtlsScaleForm(ArrayCount)
      .FormName = frm.NAME
      .NAME = "xxxFooterxxx"
      .Height = frm.Section(Access.acFooter).Height
    End With
    ArrayCount = ArrayCount + 1
    ReDim Preserve arrCtlsScaleForm(ArrayCount)
    With arrCtlsScaleForm(ArrayCount)
      .FormName = frm.NAME
      .NAME = "xxxLastScaleVertFactorxxx"
      .Height = 1
    End With
    ArrayCount = ArrayCount + 1
    ReDim Preserve arrCtlsScaleForm(ArrayCount)
    With arrCtlsScaleForm(ArrayCount)
      .FormName = frm.NAME
```

```
        .NAME = "xxxLastScaleHorzFactorxxx"
        .Height = 1
    End With
End If


Call WM_apiGetWindowRect(frm.hWnd, rectWindow) 'prepare for getting size of current window
CurrentWindowWidth = rectWindow.Right - rectWindow.Left
CurrentWindowHeight = rectWindow.Bottom - rectWindow.Top


For ArrayCount = LBound(arrCtlsScaleForm) To UBound(arrCtlsScaleForm)
    If arrCtlsScaleForm(ArrayCount).FormName = frm.NAME Then
        Select Case arrCtlsScaleForm(ArrayCount).NAME
            Case "xxxLastScaleHorzFactorxxx"
                LastScaleHorzFactor = arrCtlsScaleForm(ArrayCount).Height
            Case "xxxLastScaleVertFactorxxx"
                 LastScaleVertFactor = arrCtlsScaleForm(ArrayCount).Height
        End Select
    End If
Next


'Calculate the scaling factors and round them to 2 decimal places.
'This ensures that changes happen only in the area of 1% relative to the original size
ScaleVertFactor = Round((CurrentWindowHeight / OnOpenWindowHeight), 2)
ScaleHorzFactor = Round((CurrentWindowWidth / OnOpenWindowWidth), 2)
ScaleFontFactor = VBA.IIf(ScaleHorzFactor < ScaleVertFactor, ScaleHorzFactor, ScaleVertFactor)

If (ScaleVertFactor = LastScaleVertFactor) And (ScaleHorzFactor = LastScaleHorzFactor) Then
    'nothing to rescale
    Exit Sub
Else
    'there is something to rescale, so we update the array
    For ArrayCount = LBound(arrCtlsScaleForm) To UBound(arrCtlsScaleForm)
        If arrCtlsScaleForm(ArrayCount).FormName = frm.NAME Then
            Select Case arrCtlsScaleForm(ArrayCount).NAME
                Case "xxxLastScaleHorzFactorxxx"
                    arrCtlsScaleForm(ArrayCount).Height = ScaleHorzFactor
                Case "xxxLastScaleVertFactorxxx"
                    arrCtlsScaleForm(ArrayCount).Height = ScaleVertFactor
            End Select
        End If
    Next
```

```
End If



frm.Painting = False 'Turn off painting


For ArrayCount = LBound(arrCtlsScaleForm) To UBound(arrCtlsScaleForm)
  If arrCtlsScaleForm(ArrayCount).FormName = frm.NAME Then
    Select Case arrCtlsScaleForm(ArrayCount).NAME
      Case "xxxHeaderxxx"
        frm.Section(Access.acHeader).Height = arrCtlsScaleForm(ArrayCount).Height * ScaleVert
      Case "xxxDetailxxx"
        frm.Section(Access.acDetail).Height = arrCtlsScaleForm(ArrayCount).Height * ScaleVertF
      Case "xxxFooterxxx"
        frm.Section(Access.acFooter).Height = arrCtlsScaleForm(ArrayCount).Height * ScaleVertF
    End Select
  End If
Next ArrayCount


For ArrayCount = LBound(arrCtlsScaleForm) To UBound(arrCtlsScaleForm)
  If (arrCtlsScaleForm(ArrayCount).FormName = frm.NAME) And (arrCtlsScaleForm(ArrayCount).
<> "") Then
    'On Error GoTo err
    If frm.Controls.Item(arrCtlsScaleForm(ArrayCount).NAME).ControlType <> Access.acPage The
'Ignore pages in Tab controls.
      With frm.Controls.Item(arrCtlsScaleForm(ArrayCount).NAME)
        If ScaleVertFactor <> LastScaleVertFactor Then
          .Height = arrCtlsScaleForm(ArrayCount).Height * ScaleVertFactor
          .Top = arrCtlsScaleForm(ArrayCount).Top * ScaleVertFactor
        End If
        If ScaleHorzFactor <> LastScaleHorzFactor Then
          .Left = arrCtlsScaleForm(ArrayCount).Left * ScaleHorzFactor
          .Width = arrCtlsScaleForm(ArrayCount).Width * ScaleHorzFactor
        End If
        If .FontSize > 0 Then
          .FontSize = arrCtlsScaleForm(ArrayCount).FontSize * ScaleFontFactor
        End If
        Select Case frm.Controls.Item(arrCtlsScaleForm(ArrayCount).NAME).ControlType
          Case Access.acListBox
            .ColumnWidths = ScaleColumnWidths(arrCtlsScaleForm(ArrayCount).ColumnWidths
ScaleHorzFactor)
          Case Access.acComboBox
            .ColumnWidths = ScaleColumnWidths(arrCtlsScaleForm(ArrayCount).ColumnWidths
ScaleHorzFactor)
```

```
                .ListWidth = arrCtlsScaleForm(ArrayCount).ListWidth * ScaleHorzFactor
            Case Access.acTabCtl
                .TabFixedWidth = arrCtlsScaleForm(ArrayCount).TabFixedWidth * ScaleHorzFactor
                .TabFixedHeight = arrCtlsScaleForm(ArrayCount).TabFixedHeight * ScaleVertFactor
        End Select
      End With
    End If
  End If
Next


frm.Painting = True


'Keep this comments for debugging reasons
frm.txtOnOpenwindowWidth = OnOpenWindowWidth
frm.txtOnOpenwindowHeight = OnOpenWindowHeight
frm.txtCurrentwindowWidth = CurrentWindowWidth
frm.txtCurrentwindowHeight = CurrentWindowHeight
frm.txtscalevertfactor = ScaleVertFactor
frm.txtscalehorzfactor = ScaleHorzFactor


'Free up resources
Set ctl = Nothing 'Free up resources.
Set frm = Nothing 'Free up resources.


End Sub
Function isSubform(frmIn As Form) As Boolean

Dim strX As String

On Error Resume Next

strX = frmIn.Parent.NAME
isSubform = Err.Number = 0

End Function
Private Function ScaleColumnWidths(DesignColumnWidths As String, ScaleColumnWidthFactor As
As String

On Error Resume Next

Dim DesignColumnWidthsArray() As String
```

```
Dim NewColumnWidths As String
Dim TempVar As Long

ReDim DesignColumnWidthsArray(0)

DesignColumnWidthsArray = Split(DesignColumnWidths, ";")
NewColumnWidths = vbNullString

For TempVar = LBound(DesignColumnWidthsArray) To UBound(DesignColumnWidthsArray)
   If Not IsNull(DesignColumnWidthsArray(TempVar)) And DesignColumnWidthsArray(TempVar) Then
      NewColumnWidths = NewColumnWidths & CSng(DesignColumnWidthsArray(TempVar)) * ScaleColumnWidthFactor & ";"
   End If
Next

ScaleColumnWidths = NewColumnWidths
Erase DesignColumnWidthsArray

End Function

Function InitialInsertCodeIntoForms()

Dim lngProcCountLines As Long
Dim lngProcStartLine As Long
Dim loCont As Container
Dim loDb As Database
Dim loDoc As Document
Dim loForm As Form
Dim loMod As Module
Dim pkType As Long
Dim strCode As String
Dim strName As String


Set loDb = CurrentDb
Set loCont = loDb.Containers("Forms")


For Each loDoc In loCont.Documents
   strName = loDoc.NAME
   DoCmd.OpenForm strName, acDesign
   Set loForm = Forms(strName)
   Debug.Print loForm.NAME; ".HasModule = "; loForm.HasModule
   If loForm.HasModule = True Then
```

```
599         Set loMod = loForm.Module
600         On Error Resume Next
601         lngProcStartLine = loMod.ProcStartLine("Form_Open", pkType)
602         If Err > 0 Then
603            Debug.Print vbTab; "FormOpen Added"
604            loForm.OnOpen = "[Event Procedure]"
605            lngProcStartLine = loMod.CountOfLines
606            strCode = "Private Sub Form_Open(Cancel As Integer)" _
607            & vbCrLf _
608            & "ScaleFormWindow Me" _
609            & vbCrLf _
610            & "End Sub"
611            loMod.InsertLines lngProcStartLine + 1, strCode
612         Else
613            Debug.Print vbTab; "FormOpen Edited"
614            lngProcCountLines = loMod.ProcCountLines("Form_Open", pkType)
615            loMod.InsertLines lngProcStartLine + lngProcCountLines - 1, "ScaleFormWindow Me"
616         End If
617         If loForm.OnOpen <> "[Event Procedure]" Then Debug.Print vbTab; "Check OnOpen Event, it
    to [Event Procedure]."
618         Err.Clear
619      End If
620
621      If loForm.HasModule = True Then
622         Set loMod = loForm.Module
623         On Error Resume Next
624         lngProcStartLine = loMod.ProcStartLine("Form_Resize", pkType)
625         If Err > 0 Then
626            Debug.Print vbTab; "FormResize Added"
627            loForm.OnOpen = "[Event Procedure]"
628            lngProcStartLine = loMod.CountOfLines
629            strCode = "Private Sub Form_Resize()" _
630            & vbCrLf _
631            & "ScaleFormControls Me" _
632            & vbCrLf _
633            & "End Sub"
634            loMod.InsertLines lngProcStartLine + 1, strCode
635         Else
636            Debug.Print vbTab; "FormResize Edited"
637            lngProcCountLines = loMod.ProcCountLines("Form_Resize", pkType)
638            loMod.InsertLines lngProcStartLine + lngProcCountLines - 1, "ScaleFormControls Me"
639         End If
640         If loForm.OnResize <> "[Event Procedure]" Then Debug.Print vbTab; "Check OnResize Event,
    set to [Event Procedure]."
641         Err.Clear
```

```
    End If
    DoCmd.Close acForm, strName, acSaveYes
Next


Set loMod = Nothing
Set loForm = Nothing
Set loDoc = Nothing
Set loCont = Nothing
Set loDb = Nothing


End Function

Function PreScale()

Dim ArrayCount As Long
Dim ctl As Control
Dim DesignResDPI As Long
Dim DesignResHeight As Long
Dim DesignResWidth As Long
Dim FormHasToBeScaled As Boolean
Dim FormNameArray() As String
Dim frm As Object
Dim frm2 As Form
Dim KeepOriginalFormNames As Boolean
Dim NewFormName As String
Dim ScaleFontFactor As Single
Dim ScaleHorzFactor As Single
Dim ScaleVertFactor As Single
Dim TagString As String
Dim TagStringArray() As String
Dim TagStringPosition As Long
Dim TagStringPosition2 As Long
Dim TagStringPosition3 As Long
Dim TagStringTemp As String
Dim TargetResDPI As Long
Dim TargetResHeight As Long
Dim TargetResString As String
Dim TargetResWidth As Long
Dim TempString As String


'Define options here
```

```
TargetResWidth = 1400
TargetResHeight = 1050
TargetResDPI = 96
KeepOriginalFormNames = False


Debug.Print "=========================================="
Debug.Print "Starting"
Debug.Print "=========================================="


'Write the names of all forms to an array
ReDim FormNameArray(0)
ArrayCount = UBound(FormNameArray)

Debug.Print
Debug.Print "Writing form names to array"
Debug.Print "=========================================="
Debug.Print

For Each frm In CurrentProject.AllForms
   ReDim Preserve FormNameArray(ArrayCount)
   FormNameArray(ArrayCount) = frm.NAME
   Debug.Print frm.NAME
   ArrayCount = ArrayCount + 1
Next

TargetResString = "_mSF" & TargetResWidth & "x" & TargetResHeight & "x" & TargetResDPI

'delete all existing prescaled forms for target resolution
Debug.Print
Debug.Print "Deleting existing forms for " & TargetResWidth & "x" & TargetResHeight & "x" &
TargetResDPI
Debug.Print "=========================================="
Debug.Print

For ArrayCount = LBound(FormNameArray) To UBound(FormNameArray)
   If Right(FormNameArray(ArrayCount), Len(TargetResString)) = TargetResString Then 'this form
deleted
      On Error Resume Next
      DoCmd.DeleteObject acForm, FormNameArray(ArrayCount)
      Debug.Print FormNameArray(ArrayCount)
   End If
Next

```

```
Debug.Print
Debug.Print "Starting form manipulation"
Debug.Print "========================================="
Debug.Print


For ArrayCount = LBound(FormNameArray) To UBound(FormNameArray)
  Debug.Print FormNameArray(ArrayCount)
  If InStr(1, FormNameArray(ArrayCount), "_msf", vbTextCompare) > 0 Then
    'form has "_msf" in the name, so it has already been scaled for another resolution
    Debug.Print "   skipped, it has already been scaled for another resolution"
  Else
    If Len(FormNameArray(ArrayCount) & TargetResString) <= 64 Then
      FormHasToBeScaled = True
      DoCmd.OpenForm FormNameArray(ArrayCount), acDesign
      Set frm2 = Forms(FormNameArray(ArrayCount))
      'Extract "DesignRes:*:" from Tag-Option of Form
      TagString = LCase(frm2.Tag) 'lower case tag-field
      TagStringPosition = InStr(TagString, "designres:") 'get left position of resolution
      If TagStringPosition > 0 Then
        TagString = Mid(TagString, TagStringPosition + 10) 'get string starting with resolution (t
"+10" is the char count of "DesignRes:"
        TagStringPosition = InStr(TagString, ":") 'Get Position of ":" behind resolution
        TagString = Left(TagString, TagStringPosition - 1) 'remove everything behind ":" - the va
now contains the design resolution
        ReDim TagStringArray(1)
        TagStringArray = Split(TagString, "x") '"x" is the delimiter
          If UBound(TagStringArray) >= 0 Then DesignResWidth = TagStringArray(0)
          If UBound(TagStringArray) >= 1 Then DesignResHeight = TagStringArray(1)
          If UBound(TagStringArray) >= 2 Then DesignResDPI = TagStringArray(2)
          If (DesignResWidth <= 0) Or (DesignResHeight <= 0) Or (DesignResDPI <= 0) Then
            DesignResWidth = 0
            DesignResHeight = 0
            DesignResDPI = 0
          End If
          Erase TagStringArray
      Else
        'DesignRes is not specified in the form so we use default values for scaling
        DesignResWidth = 0
        DesignResHeight = 0
        DesignResDPI = 0
      End If
      If (DesignResWidth > 0) And (DesignResHeight > 0) And (DesignResDPI > 0) Then
        If (DesignResWidth = TargetResWidth) And (DesignResHeight = TargetResHeight) And
(DesignResDPI = TargetResDPI) Then
        Debug.Print "   Forms DesignRes is equal to the TargetRes, so it does not have to be scale
```

```
772             FormHasToBeScaled = False
773             End If
774           Else
775             If (DefaultDesignWidth = TargetResWidth) And (DefaultDesignHeight = TargetResHeigh
(DefaultDesignDPI = TargetResDPI) Then
776               Debug.Print "   Forms DesignRes is not set, TargetRes is equal to DefaultDesignRes, so
not have to be scaled"
777               FormHasToBeScaled = False
778             End If
779           End If
780           DoCmd.Close acForm, FormNameArray(ArrayCount), acSaveNo
781           If FormHasToBeScaled = True Then
782             On Error Resume Next
783             If KeepOriginalFormNames = True Then
784               NewFormName = FormNameArray(ArrayCount)
785             Else
786               NewFormName = FormNameArray(ArrayCount) & TargetResString
787               DoCmd.CopyObject , NewFormName, acForm, FormNameArray(ArrayCount)
788               Debug.Print "   copying to " & NewFormName
789               Debug.Print "   changing subform names"
790             End If
791             DoCmd.OpenForm NewFormName, acDesign
792             If KeepOriginalFormNames = False Then
793               For Each ctl In Screen.ActiveForm.Controls
794                 TempString = ctl.SourceObject & TargetResString
795                 ctl.SourceObject = TempString
796               Next
797             End If
798             Debug.Print "   scaling controls"
799             If (DesignResWidth > 0) And (DesignResHeight > 0) And (DesignResDPI > 0) Then
800               ScaleVertFactor = (TargetResHeight / DesignResHeight) * (TargetResDPI / DesignResD
801               ScaleHorzFactor = (TargetResWidth / DesignResWidth) * (TargetResDPI / DesignResDP
802               ScaleFontFactor = VBA.IIf(ScaleHorzFactor < ScaleVertFactor, ScaleHorzFactor, ScaleV
803             Else
804               ScaleVertFactor = (TargetResHeight / DefaultDesignHeight) * (TargetResDPI /
DefaultDesignDPI)
805               ScaleHorzFactor = (TargetResWidth / DefaultDesignWidth) * (TargetResDPI / DefaultD
806               ScaleFontFactor = VBA.IIf(ScaleHorzFactor < ScaleVertFactor, ScaleHorzFactor, ScaleV
807             End If
808             Screen.ActiveForm.Width = Screen.ActiveForm.Width * ScaleHorzFactor
809             Screen.ActiveForm.Section(Access.acHeader).Height =
Screen.ActiveForm.Section(Access.acHeader).Height * ScaleVertFactor
810             Screen.ActiveForm.Section(Access.acDetail).Height =
Screen.ActiveForm.Section(Access.acDetail).Height * ScaleVertFactor
811             Screen.ActiveForm.Section(Access.acFooter).Height =
Screen.ActiveForm.Section(Access.acFooter).Height * ScaleVertFactor
812             For Each ctl In Screen.ActiveForm.Controls
```

```
            If ctl.ControlType <> Access.acPage Then  'Ignore pages in Tab controls.
              With ctl
                .Height = ctl.Height * ScaleVertFactor
                .Top = ctl.Top * ScaleVertFactor
                .Left = ctl.Left * ScaleHorzFactor
                .Width = ctl.Width * ScaleHorzFactor
                If .FontSize > 0 Then
                  .FontSize = ctl.FontSize * ScaleFontFactor
                End If
              Select Case ctl.ControlType
                Case Access.acListBox
                  .ColumnWidths = ScaleColumnWidths(ctl.ColumnWidths, ScaleHorzFactor)
                Case Access.acComboBox
                  .ColumnWidths = ScaleColumnWidths(ctl.ColumnWidths, ScaleHorzFactor)
                  .ListWidth = ctl.ListWidth * ScaleHorzFactor
                Case Access.acTabCtl
                  .TabFixedWidth = ctl.TabFixedWidth * ScaleHorzFactor
                  .TabFixedHeight = ctl.TabFixedHeight * ScaleVertFactor
              End Select
            End With
            End If
          Next
          frm.Painting = True
          'remove temporary OnOpenRes from Tag
          Set frm2 = Forms(NewFormName)
          frm2.Tag = Left(frm2.Tag, (InStr(LCase(frm2.Tag), LCase("onopenres:")) - 3))
          'Update DesignRes
          If InStr(LCase(frm2.Tag), LCase("designres:")) > 0 Then 'designres is set in the form, so re
            TagStringPosition2 = InStr(LCase(frm2.Tag), LCase("designres:"))
            TagStringPosition3 = InStr(TagStringPosition2 + Len("designres:"), frm2.Tag, ":")
            TagStringPosition3 = Len(frm2.Tag) - TagStringPosition3
            TagStringTemp = Left(frm2.Tag, TagStringPosition2 - 1) & Right(frm2.Tag, TagStringP
            frm2.Tag = TagStringTemp
          End If
          frm2.Tag = frm2.Tag & ", DesignRes:" & TargetResWidth & "x" & TargetResHeight & "x'
TargetResDPI & ":"
          'Save scaled form
          DoCmd.Close acForm, NewFormName, acSaveYes
        End If
      Else
        Debug.Print NewFormName & " would have more than 64 characters - form was not create
      End If
    End If
  Next

```

```
Debug.Print
Debug.Print "==========================================="
Debug.Print "Finished!"
Debug.Print "==========================================="

End Function

Public Function mSFOpenForm( _
        FormName As String, _
        Optional View As AcFormView = acNormal, _
        Optional FilterName As String, _
        Optional WhereCondition As String, _
        Optional DataMode As AcFormOpenDataMode = acFormPropertySettings, _
        Optional WindowMode As AcWindowMode = acWindowNormal, _
        Optional OpenArgs As String) As String
        'this function returns the name of the opened form as String

Dim frm As Object
Dim GetScreenResolution As tDisplay
Dim hDCcaps As Long
Dim lngRtn As Long
Dim ScaledFormName As String


hDCcaps = WM_apiGetDC(0) 'Get display context for desktop (hwnd = 0).
With GetScreenResolution
  .Height = WM_apiGetDeviceCaps(hDCcaps, WM_VERTRES)
  .Width = WM_apiGetDeviceCaps(hDCcaps, WM_HORZRES)
  .DPI = WM_apiGetDeviceCaps(hDCcaps, WM_LOGPIXELSX)
End With
lngRtn = WM_apiReleaseDC(0, hDCcaps) 'Release display context.

ScaledFormName = FormName & "_mSF" & GetScreenResolution.Width & "x" &
GetScreenResolution.Height & "x" & GetScreenResolution.DPI
For Each frm In CurrentProject.AllForms
  If LCase(frm.NAME) = LCase(ScaledFormName) Then
    FormName = frm.NAME
  End If
Next

DoCmd.OpenForm FormName, View, FilterName, WhereCondition, DataMode, WindowMode, Ope
mSFOpenForm = FormName

End Function

```

901

902

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	20/4/2026 15:29:17
LastUpdated:	20/4/2026 15:29:17	Owner:	admin
UserName:	admin		

Código

```
Attribute VB_Name = "MÓDULO CENTRALIZADO DE VISOR PDF"
' ================================
'  MÓDULO CENTRALIZADO DE VISOR PDF
'  Compatible con Access 64 bits
'  Sin dependencias COM
'  Usa WebBrowser / WebView2
' ================================

Option Compare Database
Option Explicit

Public Sub MostrarPDF(RutaPDF As String, Optional frm As Form)

  On Error GoTo ErrHandler

  ' Si no se pasa formulario, usa el activo
  If frm Is Nothing Then
    Set frm = Screen.ActiveForm
  End If

  Dim ctrl As Control
  Dim encontrado As Boolean
  encontrado = False

  ' Buscar control WebBrowser o WebView2 automáticamente
  For Each ctrl In frm.Controls
    Select Case ctrl.ControlType

      Case acWebBrowser
        ' WebBrowser clásico (Edge/IE)
        ctrl.Object.Navigate "file:///" & Replace(RutaPDF, "\", "/")
        encontrado = True
        Exit For

      Case acCustomControl
        ' WebView2 (control moderno)
```

```
37            On Error Resume Next
38            ctrl.Object.CoreWebView2.Navigate RutaPDF
39            If Err.Number = 0 Then
40               encontrado = True
41               Exit For
42            End If
43            On Error GoTo ErrHandler
44
45         End Select
46      Next ctrl
47
48      ' Si no hay visor ? abrir en navegador externo
49      If Not encontrado Then
50         Application.FollowHyperlink RutaPDF
51      End If
52
53      Exit Sub
54
55   ErrHandler:
56      MsgBox "Error al mostrar PDF: " & Err.Description, vbExclamation, "Visor PDF"
57   End Sub
58
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	8/3/2024 19:30:50
LastUpdated:	8/3/2024 19:30:50	Owner:	admin
UserName:	admin		

Código

```
1 Attribute VB_Name = "Módulo1"
2 Option Compare Database
3
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	8/3/2024 19:30:50
LastUpdated:	8/3/2024 19:30:50	Owner:	admin
UserName:	admin		

Código

```
1 Attribute VB_Name = "Módulo2"
2 Option Compare Database
3
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	8/3/2024 19:30:50
LastUpdated:	8/3/2024 19:30:50	Owner:	admin
UserName:	admin		

Código

```
Attribute VB_Name = "Módulo3"
Option Compare Database

Sub ExportarAExcel()
   Dim db As Database
   Dim rs As Recordset
   Dim ExcelApp As Object
   Dim ExcelBook As Object
   Dim ExcelSheet As Object
   Dim i As Integer

   ' Establecer referencia a la base de datos y subformulario
   Set db = CurrentDb
   Set rs = Me.subformPresentes.Form.RecordsetClone

   ' Crear una instancia de Excel
   Set ExcelApp = CreateObject("Excel.Application")
   ExcelApp.Visible = True

   ' Crear un nuevo libro de Excel
   Set ExcelBook = ExcelApp.Workbooks.Add
   Set ExcelSheet = ExcelBook.Worksheets(1)

   ' Encabezados de columna
   For i = 1 To rs.Fields.Count
      ExcelSheet.Cells(1, i).Value = rs.Fields(i - 1).NAME
   Next i

   ' Datos del subformulario
   rs.MoveFirst
   i = 2 ' Comenzar en la segunda fila
   Do Until rs.EOF
      For j = 1 To rs.Fields.Count
         ExcelSheet.Cells(i, j).Value = rs.Fields(j - 1).Value
      Next j
      i = i + 1
```

```
        rs.MoveNext
    Loop

    ' Limpieza
    rs.Close
    Set rs = Nothing
    Set db = Nothing
End Sub

```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	8/3/2024 19:30:50
LastUpdated:	8/3/2024 19:30:50	Owner:	admin
UserName:	admin		

Código

```
1 Attribute VB_Name = "Módulo4"
2 Option Compare Database
3
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	4/4/2025 10:10:08
LastUpdated:	4/4/2025 10:10:08	Owner:	admin
UserName:	admin		

Código

```
1  Attribute VB_Name = "Módulo5"
2  Option Compare Database
3
4  Function ReemplazarValoresTexto(filtro As String) As String
5     Dim patron As Object
6     Set patron = CreateObject("VBScript.RegExp")
7
8     patron.Global = True
9     patron.IgnoreCase = False
10    patron.Pattern = "([\w]+) *= *([^0-9'""\s][^ ]*)" ' Busca campos = valores de texto
11
12    ' Agregar comillas simples a los valores de texto
13    ReemplazarValoresTexto = patron.Replace(filtro, "$1 = '$2'")
14 End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	18/5/2016 01:06:20
Description:	Functions used on the Startup form.	LastUpdated:	18/5/2016 01:06:20
Owner:	admin	UserName:	admin

Código

```
Attribute VB_Name = "Startup"
Option Compare Database
Option Explicit
' Functions in this module are used in the Startup form.

Function OpenStartup() As Boolean
' Displays Startup form only if database is not a design master or replica.
' Used in OnOpen property of Startup form.
On Error GoTo OpenStartup_Err
   If IsItAReplica() Then
      ' This database is a design master or replica, so close Startup form
      ' before it is displayed.
       DoCmd.Close
   Else
      ' This database is not a design master or replica, so display Startup form.
      ' Set the value of HideStartupForm check box using the value of
      ' StartupForm property of database (as set in Display Form box
      ' in Startup dialog box).
      If CurrentDb().Properties("StartupForm") = "Startup" Then
         ' StartupForm property is set to Startup, so clear HideStartupForm
         ' check box.
         Forms!Startup!HideStartupForm = False
      Else
         ' StartupForm property is not set to Startup, so check HideStartupForm
         ' checkbox.
         Forms!Startup!HideStartupForm = True
      End If
   End If

OpenStartup_Exit:
   Exit Function

OpenStartup_Err:
   Const conPropertyNotFound = 3270
   If Err = conPropertyNotFound Then
```

```
36        Forms!Startup!HideStartupForm = True
37        Resume OpenStartup_Exit
38     End If
39  End Function
40
41  Function HideStartupForm()
42  On Error GoTo HideStartupForm_Err
43  ' Uses the value of HideStartupForm check box to determine the setting for
44  ' StartupForm property of database. (The setting is displayed in Display Form
45  ' box in Startup dialog box).
46  ' Used in OnClose property of Startup form.
47        If Forms!Startup!HideStartupForm Then
48        ' HideStartupForm check box is checked, so set StartupForm property to (none).
49           CurrentDb().Properties("StartupForm") = "(none)"
50        Else
51           ' HideStartupForm check box is cleared, so set StartupForm property to Startup.
52           CurrentDb().Properties("StartupForm") = "Startup"
53        End If
54
55        Exit Function
56
57  HideStartupForm_Err:
58     Const conPropertyNotFound = 3270
59     If Err = conPropertyNotFound Then
60        Dim db As Database
61        Dim prop As Property
62        Set db = CurrentDb()
63        Set prop = db.CreateProperty("StartupForm", dbText, "Startup")
64        db.Properties.Append prop
65        Resume Next
66     End If
67  End Function
68  Function CloseForm()
69  ' Closes Startup form.
70  ' Used in OnClick property of OK command button on Startup form.
71     DoCmd.Close
72
73  End Function
74  Function IsItAReplica() As Boolean
75  On Error GoTo IsItAReplica_Err
76  ' Determines if database is a design master or a replica.
77  ' Used in OpenStartup function.
78
79     Dim blnReturnValue As Boolean
```

```
80
81      bInReturnValue = False
82      If CurrentDb().Properties("Replicable") = "T" Then
83          'Replicable property setting is T, so database is a design master or replica.
84          bInReturnValue = True
85      Else
86          ' Replicable property setting is not T, so database is not a design master
87          ' or replica.
88          bInReturnValue = False
89      End If
90
91  IsItAReplica_Exit:
92      IsItAReplica = bInReturnValue
93      Exit Function
94
95  IsItAReplica_Err:
96      Resume IsItAReplica_Exit
97  End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	3/2/2015 21:03:52
LastUpdated:	3/2/2015 21:03:52	Owner:	admin
UserName:	admin		

Código

```
1   Attribute VB_Name = "TipoTitulo"
2   Option Compare Database
3
4   Public Function PrimeraLetraMayusculas(strCadena As String) As String
5
6   Dim Matriz As Variant, _
7      i As Long, _
8      j As Long, _
9      strExcepciones As String, _
10     MatrizExcepciones As Variant
11
12  ' aqui se pueden añadir o quitar cuantas excepciones se deseen controlar
13  strExcepciones =
    "el,la,del,de,al,y,o,u,los,las,para,a,ante,bajo,con,contra,de,desde,en,entre,hacia,hasta,para,por,segú
    re,tras"
14
15  ' troceo las particulas en una matriz
16  MatrizExcepciones = Split(strExcepciones, ",")
17  ' troceo la cadena en una matriz
18  Matriz = Split(strCadena, " ")
19
20  ' recorro las distintas palabras a convertir
21  For i = 0 To UBound(Matriz)
22     ' comparo cada palabra con las excepciones
23     For j = 0 To UBound(MatrizExcepciones)
24        ' si la palabra es una excepción, pero no es la primera palabra de la cadena no hago nada
25        If MatrizExcepciones(j) = Matriz(i) And Not i = 0 Then GoTo Saltar
26     Next j
27     ' si la palabra no es una excepción, concateno la primera letra en mayúsculas con el resto de la pa
28     Matriz(i) = UCase(Left(Matriz(i), 1)) & Mid(Matriz(i), 2)
29  Saltar:
30  Next i
31
32  ' devuelvo la cadena convertida
33  PrimeraLetraMayusculas = Join(Matriz, " ")
34
```

```
End Function        ' PrimeraLetraMayusculas



Public Function AddTrustedLocation()
On Error GoTo err_proc
'WARNING:  THIS CODE MODIFIES THE REGISTRY
'You do not need administrator privileges
'since it only affects the HK_CURRENT_USER hive
'sets registry key for 'trusted location'

Dim intLocns As Integer
Dim i As Integer
Dim intNotUsed As Integer
Dim strLnKey As String
Dim reg As Object
Dim strPath As String
Dim strTitle As String

strTitle = "Add Trusted Location"
Set reg = CreateObject("wscript.shell")
strPath = CurrentProject.Path

'Specify the registry trusted locations path for the Access 2010 runtime
strLnKey = "HKEY_CURRENT_USER\Software\Microsoft\Office\14.0\Access\Security\Trusted Locations\Location"

On Error GoTo err_proc0
'find top of range of trusted locations references in registry
For i = 999 To 0 Step -1
   reg.RegRead strLnKey & i & "\Path"
   GoTo chckRegPths       'Reg.RegRead successful, location exists > check for path in all locations
checknext:
Next
MsgBox "Unexpected Error - No Registry Locations found", vbExclamation
   GoTo exit_proc

chckRegPths:
   'Check if Currentdb path already a trusted location
   'reg.RegRead fails before intlocns = i then the registry location is unused and
   'will be used for new trusted location if path not already in registy

   On Error GoTo err_proc1:
   For intLocns = 1 To i
      reg.RegRead strLnKey & intLocns & "\Path"
```

```
79        'If Path already in registry -> exit
80        If InStr(1, reg.RegRead(strLnKey & intLocns & "\Path"), strPath) = 1 Then GoTo exit_proc
81    NextLocn:
82      Next
83
84      If intLocns = 999 Then
85        MsgBox "Location count exceeded - unable to write trusted location to registry", vbInformati
    strTitle
86        GoTo exit_proc
87      End If
88      'if no unused location found then set new location for path
89      If intNotUsed = 0 Then intNotUsed = i + 1
90
91      'Write Trusted Location regstry key to unused location in registry
92      On Error GoTo err_proc:
93      strLnKey = strLnKey & intNotUsed & "\"
94      reg.RegWrite strLnKey & "AllowSubfolders", 1, "REG_DWORD"
95      reg.RegWrite strLnKey & "Date", Now(), "REG_SZ"
96      reg.RegWrite strLnKey & "Description", Application.CurrentProject.NAME, "REG_SZ"
97      reg.RegWrite strLnKey & "Path", strPath & "\", "REG_SZ"
98
99    exit_proc:
100       Set reg = Nothing
101       Exit Function
102
103   err_proc0:
104       Resume checknext
105
106   err_proc1:
107       If intNotUsed = 0 Then intNotUsed = intLocns
108       Resume NextLocn
109
110   err_proc:
111       MsgBox Err.Description, , strTitle
112       Resume exit_proc
113
114
115   End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	18/5/2016 01:06:20
LastUpdated:	18/5/2016 01:06:20	Owner:	admin
PublishToWeb:	1	UserName:	admin

Código

```
Attribute VB_Name = "Utilidades"
Option Compare Database
Option Explicit

Public eh As New ControlDeErrores

Public Enum StringIDEnum
    NoPropuestasInPeriod = 18
    InvalidLogin = 23
    InitializeAppData = 38
    CancelPropuestasPrompt = 48
    CancelPropuestasConfirmPrompt = 49
    CancelPropuestasSuccess = 100
End Enum


Function MsgBoxYesNo(StringID As StringIDEnum, Optional ByVal strInsert As String) As Boolean
   MsgBoxYesNo = vbYes = MsgBoxID(StringID, vbYesNo, strInsert)
End Function


Sub MsgBoxOKOnly(StringID As StringIDEnum, Optional ByVal strInsert As String)
   MsgBoxID StringID, vbOKOnly, strInsert
End Sub


Function MsgBoxID(StringID As StringIDEnum, Buttons As VbMsgBoxStyle, Optional ByVal strInse
String) As VbMsgBoxResult
   MsgBoxID = MsgBox(InsertString(StringID, strInsert), Buttons, LoadString(AppTitle))
End Function


Function LoadString(StringID As StringIDEnum) As String
   LoadString = DLookupStringWrapper("[Datos de cadena]", "Cuerdas", "[Id de cadena]=" & Stri

    ' Comprobar que se ha encontrado la cadena especificada utilizando DLookupStringWrapper.
```

```
36     ' Si llega a este punto de interrupción, compruebe que existe el Id. de la cadena en la tabla de ca
37     Debug.Assert LoadString <> ""
38   End Function
39
40
41   Function InsertString(StringID As StringIDEnum, strInsert As String) As String
42     InsertString = Replace(LoadString(StringID), "|", strInsert)
43   End Function
44
45
46   Function HasSourceCode() As Boolean
47     On Error Resume Next
48     HasSourceCode = (CurrentDb.Properties("MDE") <> "T")
49     ' Error de propiedad no encontrada
50     If Err = 3270 Then HasSourceCode = True
51     On Error GoTo 0
52   End Function
53
54
55   Function IsRuntime() As Boolean
56     IsRuntime = SysCmd(acSysCmdRuntime)
57   End Function
58
59
60   Function DebuggingSupported() As Boolean
61     DebuggingSupported = HasSourceCode() And Not IsRuntime()
62   End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Propiedades

Container:	Modules	DateCreated:	18/5/2016 01:06:20
Description:	Functions used in database objects.	LastUpdated:	18/5/2016 01:06:20
Owner:	admin	UserName:	admin

Código

```
Attribute VB_Name = "Utility Functions"
Option Compare Database
Option Explicit

Function IsLoaded(ByVal strFormName As String) As Boolean
 ' Returns True if the specified form is open in Form view or Datasheet view.

   Const conObjStateClosed = 0
   Const conDesignView = 0

   If SysCmd(acSysCmdGetObjectState, acForm, strFormName) <> conObjStateClosed Then
      If Forms(strFormName).CurrentView <> conDesignView Then
         IsLoaded = True
      End If
   End If

End Function
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición

Modificar definición

Propiedades

Container:	Modules	DateCreated:	29/8/2016 23:28:58
LastUpdated:	29/8/2016 23:28:58	Owner:	admin
UserName:	admin		

Código

```
1  Attribute VB_Name = "variasmacros"
2  Option Compare Database
3  Option Explicit
4
5  Private Sub EjecutarVariasMacros()
6
7   CdoFrenteFisico
8   CdoReversoFisico
9   CdoFrentePsiq
10  CdoReversoPsiq
11
12 End Sub
13
```

Permisos de usuario

admin	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición

Permisos de grupo

Admins	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición
Users	Eliminar; Leer permisos; Establecer permisos; Cambiar propietario, Leer definición Modificar definición